Modèle de menaces — Lidl Collect (STRIDE)

Confidentiel — Document à diffusion restreinte Référence : méthodologie STRIDE (Microsoft Threat Modeling) Périmètre : MVP Click & Collect — authentification, passage de commande, accès opérateur et administration Dernière mise à jour : 2026-04-16

Note de cadrage : Ce document présente le modèle de menaces du projet Lidl Collect. Les contre-mesures documentées constituent des **préconisations de sécurité** issues de l'analyse de risques — elles indiquent ce qui doit être appliqué avant toute mise en production. Le terme "Mesure préconisée" désigne une recommandation à destination des équipes d'implémentation, pas un constat d'audit de l'existant.

1. DIAGRAMME DE FLUX DE DONNÉES — NIVEAU 1

Acteurs externes

ACTEUR	DESCRIPTION	NIVEAU DE CONFIANCE
Client	Utilisateur final via navigateur (PWA React)	Non authentifié / CLIENT
Operator	Préparateur de commande via back-office	OPERATOR
Manager	Manager de magasin via back-office	MANAGER
Admin	Administrateur système via interface dédiée	ADMIN
Brevo	Prestataire email et SMS transactionnel	Externe — sous-traitant Art. 28

Processus internes

PROCESSUS	DESCRIPTION
P1 — Nginx	Reverse proxy — terminaison TLS, distribution des requêtes, cache des assets statiques
P2 — NestJS API	Cœur applicatif — logique métier, guards JWT, validation des DTOs
P3 — Module Auth	Émission et vérification des tokens JWT, rotation des refresh tokens
P4 — Module Commande	Gestion des commandes, créneaux, substitutions, stocks
P5 — Module Admin	Gestion des comptes, audit, performance, planning

Stockages

STOCKAGE	DESCRIPTION	LOCALISATION
S1 — PostgreSQL / Supabase	Base de données principale — toutes les données personnelles	Frankfurt, UE
S2 — Cache Nginx	Assets statiques uniquement (JS, CSS, images)	Infrastructure locale
S3 — Logs Nginx	Adresses IP, user-agents — durée : 1 an	Infrastructure locale
S4 — Logs NestJS	Logs applicatifs — aucun email en clair autorisé	Infrastructure locale

Frontières de confiance (Trust Boundaries)

```
┌──────────────────────────────────────────────────────────────┐
│  INTERNET (zone non fiable)                                  │
│                                                              │
│  [Client]  [Operator]  [Manager]  [Admin]                    │
│      │         │           │         │                      │
│  ────┴─────────┴───────────┴─────────┴──── TB1 ──────────    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  ZONE APPLICATIVE (DMZ)                              │    │
│  │                                                      │    │
│  │  [P1 — Nginx]                                        │    │
│  │      │ /api/*  →  P2 NestJS                         │    │
│  │      │ /       →  assets statiques (S2 cache)       │    │
│  │  ────┴──────────────────────────── TB2 ──────────    │    │
│  │                                                      │    │
│  │  ┌──────────────────────────────────────────────┐    │    │
│  │  │  ZONE BACKEND                                │    │    │
│  │  │                                              │    │    │
│  │  │  [P2 — NestJS] ── [P3 Auth] ── [P4 Commande]     │    │    │
│  │  │      │                         [P5 Admin]        │    │    │
│  │  │  ────┴──────────────────────────── TB3 ──────    │    │    │
│  │  │                                              │    │    │
│  │  │  [S1 — PostgreSQL / Supabase — Frankfurt]    │    │    │
│  │  │                                              │    │    │
│  │  └──────────────────────────────────────────────┘    │    │
│  │                                                      │    │
│  │  [P2] ──── TB4 ──────► [Brevo — AWS Irlande EEE]     │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

FRONTIÈRE	DESCRIPTION
TB1	Internet ↔ Nginx — point d'entrée unique, terminaison TLS
TB2	Nginx ↔ NestJS — séparation proxy/applicatif
TB3	NestJS ↔ PostgreSQL — accès base de données
TB4	NestJS ↔ Brevo — appel API externe (email et SMS)

2. ASSETS À PROTÉGER — HIÉRARCHIE PAR CRITICITÉ

CRITICITÉ	ASSET	JUSTIFICATION
🔴 Critique	Données personnelles clients (Client, Client_Account)	RGPD Art. 17 — effacement obligatoire à la demande
🔴 Critique	Tokens JWT (access + refresh)	Vol = usurpation d'identité complète
🔴 Critique	Variable d'environnement `JWT_SECRET`	Compromission = forge de tokens pour tous les rôles
🔴 Critique	Variable d'environnement `HMAC_SECRET` (sel pseudonymisation)	Compromission = ré-identification des données pseudonymisées
🔴 Critique	Rotation non coordonnée du sel `HMAC_SECRET`	Une rotation sans remappage préalable rend orphelins tous les tokens pseudonymisés dans Order, AuditLog, Consent, Schedule, Performance — intégrité irréversiblement compromise
🔴 Critique	Mots de passe hashés (Bcrypt)	Cible prioritaire d'une exfiltration de base
🟠 Élevé	Table Order + OrderItem	Données comptables — conservation 10 ans
🟠 Élevé	Table Performance (préparateurs)	Données RH — base légale Art. 6.1.f
🟠 Élevé	Table AuditLog	Intégrité des traces — preuve légale
🟠 Élevé	Table Consent	Preuve de consentement — conservation durée indéterminée
🟡 Modéré	Catalogue produits + stock	Données métier — pas de données personnelles
🟡 Modéré	Table Schedule (plannings)	Données RH — conservation 5 ans après départ
🟢 Faible	Assets statiques (JS, CSS, images)	Publics par nature

3. ANALYSE STRIDE — FLUX 1 : AUTHENTIFICATION

Flux modélisé :

```
Client → POST /api/auth/login → P1 Nginx → P2 NestJS → P3 Auth → S1 PostgreSQL
                                                                        ↓
Client ← Cookie httpOnly (JWT access + refresh) ————————————————————————
```

S — Spoofing (Usurpation d'identité)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
S1.1	**Attaque par force brute** sur `POST /api/auth/login` (credential stuffing, liste de mots de passe)	🔴	Rate limiting Nginx (`limit_req_zone`) + `ThrottlerGuard` NestJS + lockout temporaire après 5 tentatives échouées
S1.2	**Vol de refresh token** par script malveillant (XSS)	🔴	Cookie `httpOnly` + `SameSite=Strict` + flag `Secure` — JavaScript ne peut pas lire le token
S1.3	**JWT forgé avec** `alg: none` — token sans signature accepté par le serveur	🔴	Rejet explicite de `alg: none` dans la configuration JWT NestJS — `algorithms: ['HS256']` — rejet de `alg: none` imposé dans tous les cas
S1.4	**Manipulation du champ** `kid` (JWT Key ID injection) — pointage vers une clé contrôlée par l'attaquant	🟠	Valider que `kid` correspond à une clé gérée en interne — ne jamais charger une clé dynamiquement depuis le payload

T — Tampering (Altération)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
T1.1	**Modification du payload JWT** côté client (champs `role` , `storeId` , `userId`)	🔴	Le rôle est relu en base de données à chaque requête — le token ne sert qu'à l'identification, pas à l'autorisation seule
T1.2	**Interception et modification en transit** (Man-in-the-Middle sur connexion non chiffrée)	🔴	HTTPS obligatoire — TLS terminé en TB1 (Nginx) — aucune route HTTP en clair
T1.3	**Mass assignment** — champ `role` passé dans le corps de `POST /api/auth/register`	🟠	`ValidationPipe` avec `whitelist: true` + `forbidNonWhitelisted: true` — le rôle est assigné par défaut (CLIENT), jamais depuis le corps de la requête

R — Repudiation (Répudiation)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
R1.1	Un utilisateur nie avoir tenté une connexion frauduleuse ou effectué des actions après authentification	🟠	`AuditLog` : `LOGIN_SUCCESS` , `LOGIN_FAILED` avec `actor_id` , timestamp, adresse IP — conservation 1 an (Art. L34-1)

I — Information Disclosure (Divulgation d'information)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
I1.1	**Message d'erreur différencié** — "email inconnu" vs "mot de passe incorrect" permet l'énumération des comptes	🟠	Réponse générique uniforme : `401 – Identifiants incorrects` dans tous les cas
I1.2	**Email en clair dans les logs NestJS** — exfiltration possible si logs compromis	🟠	Interdiction formelle de logger les emails ou mots de passe dans les logs applicatifs
I1.3	**Payload JWT lisible** (base64 non chiffré) — exposition de `userId`, `role`, `storeId`	🟡	Données personnelles exclues du payload : `userId`, `role`, `storeId` uniquement — aucun nom, email ou téléphone
I1.4	**Compromission du compte Brevo** — accès aux emails transactionnels (liens de réinitialisation, confirmations d'effacement Art. 17, codes de vérification d'identité hors-session)	🟠	DPA Brevo à signer avant production — rotation des clés API Brevo + surveillance des envois anormaux

D — Denial of Service (Déni de service)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
D1.1	**Flood sur** `/api/auth/login` — épuisement du pool de connexions PostgreSQL	🟠	Rate limiting Nginx + pool de connexions limité côté TypeORM (`max: 10`)
D1.2	**Émission massive de refresh tokens** — saturation de la table de rotation	🟡	Révocation des tokens précédents à chaque renouvellement (rotation stricte) — un seul refresh token valide par session

E — Elevation of Privilege (Élévation de privilèges)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
E1.1	**Injection** `alg: none` — token forgé sans signature permettant d'obtenir le rôle ADMIN	🔴	Rejet explicite en configuration JWT (voir S1.3)
E1.2	**Mass assignment sur** `/api/user/me` **(PATCH)** — modification du champ `role` par un CLIENT	🔴	Champ `role` exclu de la liste blanche du DTO de mise à jour — seul un ADMIN peut modifier un rôle

4. ANALYSE STRIDE — FLUX 2 : PASSAGE DE COMMANDE

Flux modélisé :

```
Client → POST /api/orders → P1 Nginx → P2 NestJS (JwtGuard + RolesGuard)
                                                ↓
                                        P4 Module Commande
                                                ↓
                                  S1 PostgreSQL (BEGIN TRANSACTION)
                                  ├── Vérification stock → UPDATE Stock
                                  ├── Réservation créneau → UPDATE Slot
                                  └── INSERT Order + OrderItems
                                                ↓
                                       COMMIT → Notification Brevo
```

S — Spoofing (Usurpation d'identité)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
S2.1	**Usurpation de** `clientId` — commande passée au nom d'un autre client en injectant un `clientId` dans le corps de la requête	🔴	Le `clientId` est systématiquement déduit de `req.user.id` (JWT vérifié) — jamais lu depuis le corps de la requête

T — Tampering (Altération)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
T2.1	**Manipulation du prix** — le client envoie un prix modifié pour un article	🔴	Le prix est calculé côté serveur depuis la table `Product` en base — le corps de la requête ne contient que l'identifiant du produit et la quantité
T2.2	**Injection SQL via paramètre non validé** (orderId, storeId, productId)	🟠	TypeORM avec requêtes paramétrées + `ParseIntPipe` sur tous les paramètres d'URL numériques
T2.3	**Race condition (double-submit)** — deux requêtes simultanées pour la même commande → stock négatif	🟠	Transaction PostgreSQL avec `SELECT FOR UPDATE` sur le stock avant décrémentation — une seule commande aboutit
T2.4	**Modification du statut d'une commande par un CLIENT** (ex. `PICKED_UP` auto-déclaré)	🟠	Transition de statut autorisée uniquement pour OPERATOR, MANAGER, ADMIN — `RolesGuard` sur `PATCH /api/orders/:id/status`

R — Repudiation (Répudiation)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
R2.1	Un client nie avoir passé une commande ou effectué une annulation	🟠	`AuditLog` : `ORDER_CREATED`, `ORDER_CANCELLED` avec `actor_id` + `order_id` + timestamp

I — Information Disclosure (Divulgation d'information)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
I2.1	**IDOR sur** `GET /api/orders/:id` — un CLIENT accède à la commande d'un autre client	🔴	Vérification systématique : `order.clientId === req.user.id` — `403 Forbidden` sinon
I2.2	**Fuite inter-magasins** — un OPERATOR accède aux commandes d'un autre magasin	🔴	`StoreOwnershipGuard` : `req.user.storeId === order.storeId` — `403 Forbidden` sinon
I2.3	**Exposition des données de commande dans le cache Nginx** — réponse API authentifiée servie à un autre utilisateur	🔴	⚠️ **Recommandation prioritaire — à implémenter avant mise en production** : `Cache-Control: private, no-store` sur toutes les routes API protégées (intercepteur NestJS global) + `proxy_no_cache 1` dans le bloc `/api/` Nginx
I2.4	**IDOR sur SubstitutionProposal** — un CLIENT accepte ou consulte une substitution appartenant à la commande d'un autre client via `substitution_id` prévisible	🟠	Vérification `substitution.order.clientId === req.user.id` avant toute lecture ou action sur une substitution

D — Denial of Service (Déni de service)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
D2.1	**Soumission massive de commandes** — épuisement des créneaux de retrait disponibles	🟠	Rate limiting par utilisateur (`ThrottlerGuard` spécifique sur `POST /api/orders`) + maximum de commandes actives par compte
D2.2	**Panier volumineux** (milliers d'articles) — délai de traitement excessif en base	🟡	Limite côté validation DTO : nombre maximum d'articles par commande

E — Elevation of Privilege (Élévation de privilèges)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
E2.1	**OPERATOR tente** `POST /api/orders` (interdit par la matrice RBAC)	🟠	`@Roles('CLIENT', 'ADMIN')` sur `POST /api/orders` — rôles OPERATOR et MANAGER exclus

5. ANALYSE STRIDE — FLUX 3 : ACCÈS ADMIN ET OPÉRATEUR

Flux modélisé :

```
Admin → GET/PATCH /api/admin/* → P1 Nginx → P2 NestJS (JwtGuard + RolesGuard ADMIN)
                                                        ↓
                                                P5 Module Admin → S1 PostgreSQL

Operator → GET/PATCH /api/operator/* → NestJS (JwtGuard + RolesGuard OPERATOR)
                                                ↓
                                        StoreOwnershipGuard (storeId) → S1 PostgreSQL
```

S — Spoofing (Usurpation d'identité)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
S3.1	**Token CLIENT avec `role: ADMIN` modifié** — tentative d'accès aux routes `/api/admin/*`	🔴	Rôle re-vérifié en base de données à chaque requête — `RolesGuard` ne fait pas confiance au payload seul
S3.2	**Vol du cookie de session ADMIN** — session hijacking	🔴	Refresh token ADMIN court (4 heures) + `httpOnly` + `SameSite=Strict` + `Secure` + MFA obligatoire à la connexion
S3.3	**Vol du cookie de session OPERATOR ou MANAGER** — session hijacking sur rôles à accès étendu (commandes magasin, stock, planning, performance)	🔴	MFA obligatoire OPERATOR et MANAGER — refresh token 8 heures + `httpOnly` + `SameSite=Strict` + `Secure`

T — Tampering (Altération)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
T3.1	**Injection de `storeId` dans le corps d'une requête OPERATOR** — accès aux données d'un autre magasin	🔴	`storeId` déduit de `req.user.storeId` (JWT), jamais du corps — `StoreOwnershipGuard` systématique sur toutes les routes OPERATOR
T3.2	**ADMIN modifie le rôle d'un autre ADMIN** sans traçabilité	🟠	`AuditLog` : `ADMIN_ROLE_MODIFIED` avec `actor_id` + `target_id` + ancien rôle + nouveau rôle

R — Repudiation (Répudiation)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
R3.1	Un ADMIN nie avoir désactivé un compte ou modifié des données sensibles	🟠	`AuditLog` exhaustif : `ACCOUNT_DISABLED`, `STOCK_MODIFIED`, `ROLE_MODIFIED` avec `actor_id` + timestamp — conservation 1 an
R3.2	Un OPERATOR nie avoir modifié le statut d'une commande	🟠	`AuditLog` : `ORDER_STATUS_CHANGED` avec `actor_id` + ancien statut + nouveau statut

I — Information Disclosure (Divulgation d'information)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
I3.1	**Endpoint `/api/admin/*` accessible sans guard** — misconfiguration NestJS (guard non appliqué sur un module)	🔴	Guard global appliqué au niveau `AppModule` + tests de non-régression : scénario CLIENT → `GET /api/admin/users` doit retourner `403`
I3.2	**Logs d'audit accessibles sans authentification** — exposition de l'activité interne	🔴	Route `GET /api/admin/audit-logs` strictement ADMIN + aucune route publique exposant les logs
I3.3	**Données de performance préparateurs accessibles par un CLIENT**	🟠	Table `Performance` — accès MANAGER et ADMIN uniquement (`RolesGuard`)

D — Denial of Service (Déni de service)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
D3.1	**ADMIN compromis procède à une désactivation massive de comptes clients**	🟠	MFA obligatoire ADMIN + alerte sur volume anormal d'actions admin dans un laps de temps court
D3.2	**Tentative de suppression des AuditLogs** par un ADMIN	🟠	Table `AuditLog` en append-only — aucune route `DELETE /api/admin/audit-logs` — suppression physique interdite par politique

E — Elevation of Privilege (Élévation de privilèges)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
E3.1	**MANAGER tente d'accéder à l'interface admin** (`/api/admin/*`)	🟠	`@Roles('ADMIN')` strict sur toutes les routes admin — MANAGER exclu
E3.2	**OPERATOR s'attribue le rôle MANAGER** via `PATCH /api/user/me` (mass assignment)	🔴	Seul un ADMIN peut modifier le rôle d'un utilisateur — champ `role` absent de la liste blanche du DTO de mise à jour CLIENT

6. SURFACE D'ATTAQUE — ENDPOINTS EXPOSÉS

Zone publique (non authentifiée)

MÉTHODE	ROUTE	RÔLES	RISQUES PRIORITAIRES
POST	`/api/auth/register`	PUBLIC	S1.1 (énumération emails), T1.3 (mass assignment role)
POST	`/api/auth/login`	PUBLIC	S1.1 (force brute), S1.3 (alg:none), I1.1 (énumération comptes)
POST	`/api/auth/refresh`	PUBLIC (cookie)	S1.2 (vol refresh token), S1.4 (kid injection)
GET	`/api/products`	PUBLIC	—
GET	`/api/products/:id`	PUBLIC	T2.2 (injection paramètre)
GET	`/api/stores`	PUBLIC	—
GET	`/api/stores/:id/slots`	PUBLIC	D2.1 (flood créneaux)

Zone CLIENT (authentifiée)

MÉTHODE	ROUTE	RÔLES	RISQUES PRIORITAIRES
GET	`/api/user/me`	CLIENT+	I2.1 (IDOR)
PATCH	`/api/user/me`	CLIENT+	T1.3 (mass assignment), E3.2 (rôle)
DELETE	`/api/user/me`	CLIENT, OPERATOR, MANAGER, ADMIN	Flux d'effacement différent selon le rôle — pour OPERATOR/MANAGER : pseudonymisation de Schedule et Performance en sus
GET	`/api/user/export`	CLIENT+	Endpoint d'exfiltration maximale : retourne email, nom, téléphone, adresse, commandes complètes, fidélité — I2.1 (IDOR si clientId en paramètre) — rate limiting requis — `AuditLog` : `EXPORT_REQUESTED` obligatoire
POST	`/api/auth/logout`	CLIENT+	—
GET	`/api/cart`	CLIENT, ADMIN	I2.1 (IDOR)
POST	`/api/cart/items`	CLIENT, ADMIN	T2.1 (prix manipulé)
PATCH	`/api/cart/items/:id`	CLIENT, ADMIN	T2.2 (injection)
DELETE	`/api/cart/items/:id`	CLIENT, ADMIN	I2.1 (IDOR)
POST	`/api/orders`	CLIENT, ADMIN	S2.1 (clientId), T2.1 (prix), T2.3 (race condition)
GET	`/api/orders`	CLIENT (les siennes)	I2.1 (IDOR)
GET	`/api/orders/:id`	CLIENT (les siennes)	I2.1 (IDOR — vecteur principal)
DELETE	`/api/orders/:id`	CLIENT (les siennes)	I2.1 (IDOR)
PATCH	`/api/orders/:id/substitutions/:subId`	CLIENT, ADMIN	I2.1 (IDOR)

Zone OPERATOR / MANAGER (authentifiée + storeId)

MÉTHODE	ROUTE	RÔLES	RISQUES PRIORITAIRES
GET	`/api/orders` (magasin)	OPERATOR, MANAGER, ADMIN	I2.2 (fuite inter-magasins)
PATCH	`/api/orders/:id/status`	OPERATOR, MANAGER, ADMIN	T2.4 (statut non autorisé)
POST	`/api/orders/:id/substitutions`	OPERATOR, MANAGER, ADMIN	I2.2 (storeId)
GET	`/api/stock`	OPERATOR, MANAGER, ADMIN	I2.2 (storeId)
PATCH	`/api/stock/:id`	MANAGER, ADMIN	T3.1 (storeId injecté)
GET	`/api/schedules`	OPERATOR (le sien), MANAGER, ADMIN	I3.3 (données RH)
GET	`/api/performance`	MANAGER, ADMIN	I3.3 (données RH préparateurs)
GET	`/api/slots`	OPERATOR, MANAGER, ADMIN	—
POST	`/api/slots`	MANAGER, ADMIN	—

Zone ADMIN (authentifiée + MFA)

MÉTHODE	ROUTE	RÔLES	RISQUES PRIORITAIRES
GET	`/api/admin/users`	ADMIN	I3.1 (guard manquant), D3.1
GET	`/api/admin/users/:id`	ADMIN	I3.1
PATCH	`/api/admin/users/:id`	ADMIN	T3.2 (modification rôle), R3.1
DELETE	`/api/admin/users/:id`	ADMIN	R3.1 (traçabilité)
GET	`/api/admin/audit-logs`	ADMIN	I3.2
GET	`/api/admin/products`	ADMIN	—
POST	`/api/admin/products`	ADMIN	—
PATCH	`/api/admin/products/:id`	ADMIN	—

6.BIS FLUX 4 — CANAUX DE NOTIFICATION ET RETRAIT PHYSIQUE

Vecteurs couverts : abus de logique métier, SMS/email, QR code de retrait.

> Ces vecteurs sont absents des flux STRIDE 1–3 car ils ne correspondent pas à une couche technique classique. Ils sont propres à la logique applicative Click & Collect.

Abus de logique métier (Business Logic Abuse)

#	SCÉNARIO	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
BL1	**Annulation frauduleuse** — CLIENT annule la commande d'un autre via IDOR sur `DELETE /api/orders/:id`	🔴	Vérification systématique `order.clientId === req.user.id` avant toute mutation
BL2	**Auto-validation de retrait** — CLIENT passe son statut à `PICKED_UP` via `PATCH /api/orders/:id/status`	🟠	Transition `PICKED_UP` autorisée uniquement pour OPERATOR, MANAGER, ADMIN — guard `RolesGuard` sur ce statut spécifiquement
BL3	**Épuisement de stock par commandes fantômes** — CLIENT crée de nombreuses commandes puis les annule pour bloquer la disponibilité	🟠	Limite de commandes actives par compte (`max_active_orders` : 3) + rate limiting sur `POST /api/orders`
BL4	**Manipulation du montant final** — CLIENT tente de soumettre un prix modifié dans le corps de la requête de commande	🔴	Prix recalculé côté serveur depuis la table `Product` — le corps de la requête n'accepte que l'identifiant du produit et la quantité

Canal de notification email et SMS (Brevo)

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
N1	**Compromission du compte Brevo** — un attaquant accède au dashboard Brevo et lit les emails transactionnels (liens Art. 17 hors-session, codes OTP)	🟠	Clés API Brevo stockées en variable d'environnement, jamais en dur — rotation régulière — alertes sur volume anormal d'envois
N2	**Énumération de numéros de téléphone** via les erreurs de l'API SMS	🟡	Réponse générique en cas d'erreur d'envoi SMS — ne pas distinguer "numéro invalide" de "envoi échoué" dans la réponse API
N3	**Injection de contenu dans les templates de notification** — contenu non sanitisé inséré dans le corps d'un email ou SMS	🟠	Sanitisation de toutes les variables insérées dans les templates Brevo — aucun champ libre de l'utilisateur directement dans le template
N4	**Phishing via fausse notification Lidl Collect** — email spoofé imitant une notification de commande	🟡	Configuration SPF, DKIM, DMARC sur le domaine expéditeur — obligatoire avant mise en production

QR code de retrait et validation physique

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
Q1	**QR code prévisible ou devinable** — token de collecte généré côté client ou avec entropie insuffisante	🔴	Token de collecte généré côté serveur avec CSPRNG (module `crypto` Node.js) — UUID v4 ou équivalent — jamais calculé côté client
Q2	**Réutilisation du QR code** — token valide après retrait effectué	🟠	Token invalidé à la première validation OPERATOR (`PICKED_UP`) — statut `USED` en base, toute tentative de validation ultérieure retourne `409 Conflict`
Q3	**Forgé à partir du QR code d'une autre commande** — devinette par incrémentation d'un identifiant séquentiel	🔴	Token opaque (non basé sur l' `orderId` séquentiel) — lien token ↔ commande uniquement en base, jamais dans le token lui-même
Q4	**Validation sans vérification d'identité** — quelqu'un récupère la commande d'un autre en présentant un QR code obtenu ou deviné	🟡	Pour les commandes de valeur élevée ou les produits réglementés (alcool) : vérification visuelle d'identité par l'employé Lidl — politique opérationnelle hors périmètre applicatif

IDOR sur données employés

#	MENACE	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
EE1	**CLIENT accède aux scores de performance d'un préparateur** via IDOR sur `GET /api/performance/:preparerId`	🟠	`RolesGuard` : route accessible Manager et Admin uniquement — OPERATOR n'accède qu'à ses propres scores via un endpoint séparé `/api/performance/me`
EE2	**OPERATOR accède aux données de performance d'un autre préparateur** du même magasin	🟠	Vérification `performance.preparerId === req.user.id` pour le rôle OPERATOR — les scores des autres membres de l'équipe sont réservés au Manager

7. VECTEURS D'ESCALADE DE PRIVILÈGES — SYNTHÈSE

#	VECTEUR	CRITICITÉ	CONTRE-MESURE PRÉCONISÉE
V1	**IDOR** — accès à une ressource appartenant à un autre utilisateur via identifiant prévisible	🔴	`order.clientId === req.user.id` sur tout endpoint retournant une ressource par identifiant
V2	`alg: none` **JWT** — token forgé sans signature	🔴	`algorithms: ['HS256']` — rejet de `alg: none` imposé dans tous les cas
V3	**JWT** `kid` **injection** — chargement d'une clé externe contrôlée par l'attaquant	🟠	Validation que `kid` correspond à une clé connue et statique — aucun chargement dynamique
V4	**Endpoint admin non protégé** — route `/api/admin/*` sans guard actif	🔴	Guard global + scénario de test : CLIENT → `GET /api/admin/users` → `403` obligatoire
V5	**Violation storeId** — OPERATOR accédant aux données d'un autre magasin	🔴	`StoreOwnershipGuard` : `user.storeId === resource.storeId` — appliqué sur toutes les routes OPERATOR
V6	**Mass assignment** — champ `role` ou `storeId` modifiable via `PATCH`	🔴	`ValidationPipe` avec `whitelist: true` — champs `role` et `storeId` absents de tous les DTOs CLIENT
V7	**Race condition** — double-submit de commande → stock négatif	🟠	Transaction PostgreSQL avec `SELECT FOR UPDATE` sur le stock
V8	**Rotation non coordonnée du sel HMAC_SECRET** — tokens pseudonymisés orphelins dans toutes les tables sous obligation légale	🔴	Remappage complet des tokens avant toute rotation — coordination obligatoire entre équipe BDD et équipe sécurité

8. SYNTHÈSE DES RISQUES PAR CRITICITÉ

Risques critiques 🔴 — Traitement prioritaire avant mise en production

RISQUE	FLUX	COMPOSANT CONCERNÉ
Force brute sur l'authentification	Flux 1	Module Auth
`alg: none` JWT	Flux 1	Module Auth
Vol de refresh token (cookie mal configuré)	Flux 1	Module Auth
Mass assignment `role`	Flux 1 + 3	Backend applicatif
IDOR sur les commandes	Flux 2	Backend applicatif
Fuite inter-magasins (storeId)	Flux 2 + 3	Backend applicatif
Cache Nginx sur routes API authentifiées	Flux 2 + 3	Infrastructure serveur + Backend
Endpoint admin sans guard	Flux 3	Backend applicatif
Elevation via storeId injecté dans le corps	Flux 3	Backend applicatif

Risques élevés 🟠 — Traitement avant v1.0

RISQUE	FLUX	COMPOSANT CONCERNÉ
Messages d'erreur différenciés (énumération comptes)	Flux 1	Backend applicatif
Injection SQL via paramètre non validé	Flux 2	Backend applicatif
Race condition stock négatif	Flux 2	Backend applicatif
JWT `kid` injection	Flux 1	Module Auth
Vol de session ADMIN	Flux 3	Module Auth
Vol de session OPERATOR / MANAGER (MFA obligatoire)	Flux 3	Module Auth
Désactivation massive de comptes (ADMIN compromis)	Flux 3	Module Auth + Infrastructure réseau
Compromission compte Brevo (accès flux Art. 17 hors-session)	Flux 1	Infrastructure réseau
Rotation non coordonnée du sel HMAC_SECRET	Transversal	Opérationnel

Risques modérés 🟡 — Traitement en v1.x

RISQUE	FLUX	COMPOSANT CONCERNÉ
Logs NestJS incluant des emails en clair	Flux 1	Backend applicatif
Flood sur les créneaux de retrait	Flux 2	Infrastructure réseau
Rotation des refresh tokens non stricte	Flux 1	Module Auth
IDOR sur SubstitutionProposal	Flux 2	Backend applicatif
Rate limiting absent sur `GET /api/user/export`	Flux 2	Backend applicatif + Infrastructure réseau

9. RECOMMANDATIONS PAR COMPOSANT

Module d'authentification

- Rejet explicite de `alg: none` dans la configuration JWT NestJS
- Validation du champ `kid` — clé statique, jamais chargée dynamiquement
- Rate limiting et lockout sur `/api/auth/login` (5 tentatives → blocage temporaire)
- Cookies : `httpOnly`, `SameSite=Strict`, `Secure` — rotation stricte des refresh tokens
- MFA obligatoire OPERATOR, MANAGER, ADMIN
- Durées de session : CLIENT 15 min / 7 j — OPERATOR 15 min / 8 h — MANAGER 15 min / 8 h — ADMIN 10 min / 4 h

Infrastructure réseau

- HTTPS obligatoire sur toutes les routes — aucune route HTTP en clair en production
- Rate limiting Nginx sur les routes publiques (authentification, catalogue)
- Segmentation réseau : base de données PostgreSQL non exposée directement sur Internet (TB3)
- Surveillance du volume d'actions admin (alertes sur anomalies)
- Rotation des clés API Brevo + surveillance des envois anormaux
- Configuration SPF, DKIM, DMARC sur le domaine expéditeur
- DPA Brevo à signer avant mise en production

Backend applicatif (NestJS)

- `ValidationPipe` global : `whitelist: true`, `forbidNonWhitelisted: true`, `transform: true`
- `RolesGuard` sur toutes les routes — aucune route sans guard explicite
- `StoreOwnershipGuard` sur toutes les routes OPERATOR et MANAGER
- Vérification `resource.clientId === req.user.id` sur tout endpoint par identifiant
- `Cache-Control: private, no-store` via intercepteur global sur toutes les routes JWT-protégées
- Interdiction d'email en clair dans les logs applicatifs
- Transaction PostgreSQL avec `SELECT FOR UPDATE` sur les opérations de stock

Infrastructure serveur (Nginx)

- Deux blocs Nginx distincts : `/` (cache activé pour les assets statiques) et `/api/` (`proxy_no_cache 1`)
- Aucune donnée de réponse API authentifiée ne doit transiter par le cache serveur